图论基础

李绍澄/王百汇

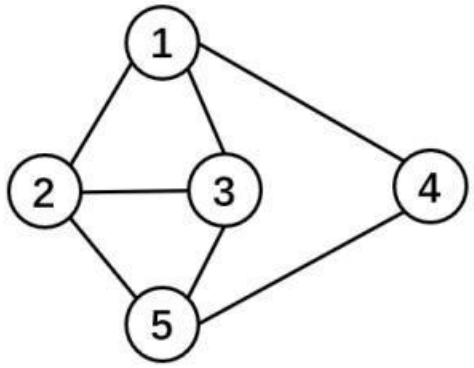
NEUACM

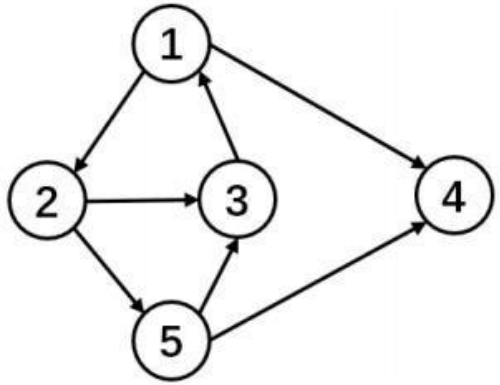
2026年1月

图的概念

图是什么？

- 图是由顶点 V的集合和边 E的集合组成的二元组
- 记：G=（V，E）。
- 存在结点v 可能含有多个前驱与多个后继

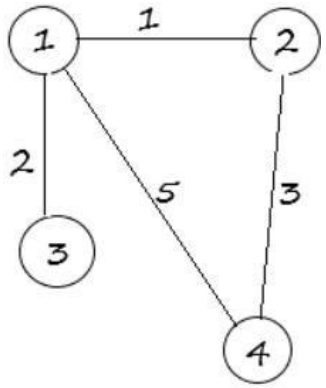




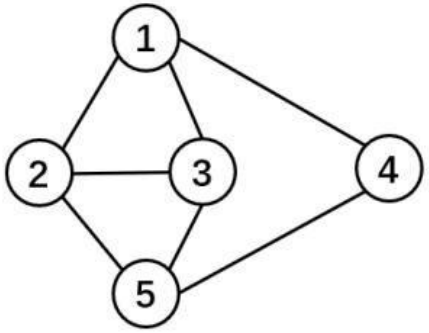
图的种类

按照不同的区分方法有以下几组分类

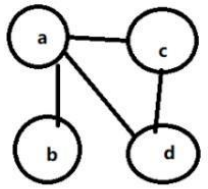
- 有向图，无向图
- 带权图，无权图
- 连通图
- 二分图
- 完全图
- 菊花图，轮图

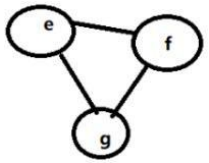


带权图

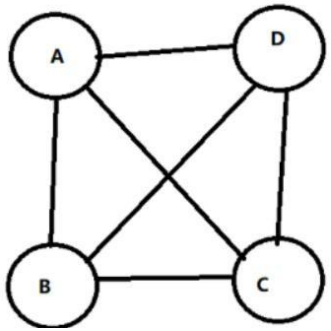


无权图

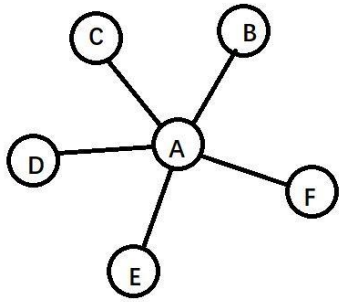




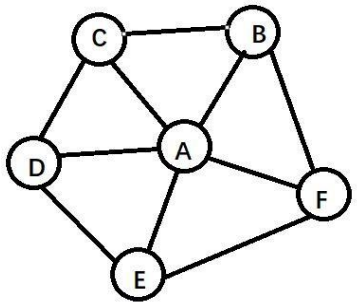
非连通图



完全图



菊花图



轮图

路径

在图 $G=(V,E)$ 中，如果结点 a，b 存在满足下述条件的结点序列 $x_1 \cdots x_k (k>1)$

(1) $x_1 = a, x_k = b$
(2) $(x_i, x_{i+1}) \in E \ (i = 1, \cdots, k-1)$

则称结点序列 $x_1 = a, x_2, \cdots, x_k = b$ 为结点 a 到结点 b 的一条路径，而路径上边的数目 $(k-1)$ 称为该路径的长度。

起点和终点相同的路径称为环（回路）

二分图

定义：一个无向图，可以分成两个点集合，保证任何两个在同一集合的点没有连边，那么这个无向图可以被称为二分图

判断二分图的方法：二分图染色（没有奇环一定是二分图）

二分图的性质：没有奇环

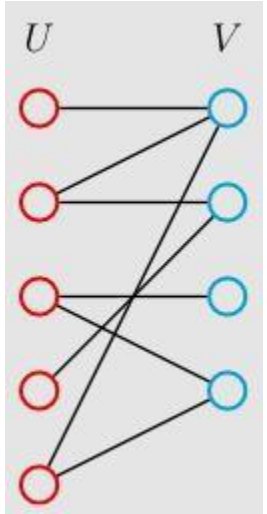
证明：因为每一条边都是从一个集合走到另一个集合，

只有走偶数次才可能回到同一个集合。

二分图是图论里一个比较重要的模型，有关二分图的

算法比较多，想要深入了解可以浏览下面的链接

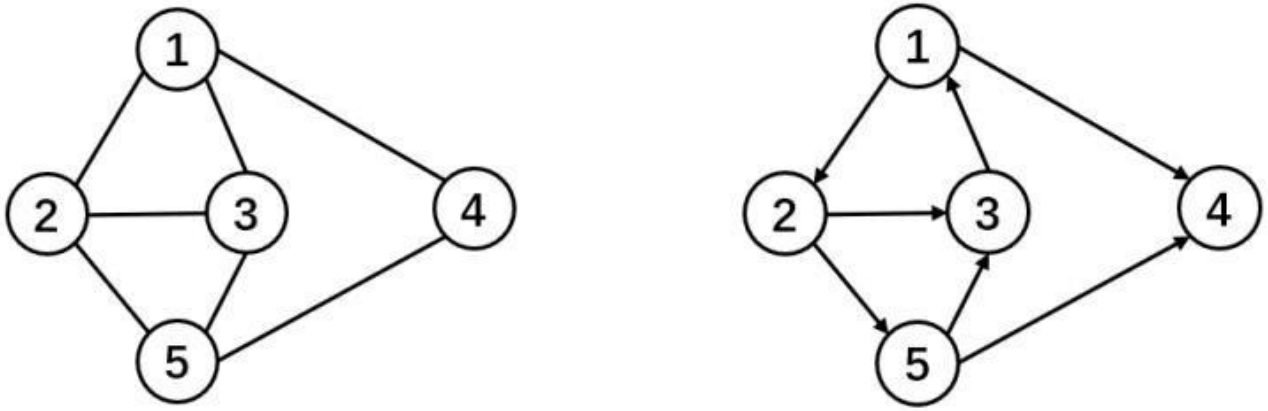
https://oiwiki.com/graph/bi-graph/



度

在无向图中，顶点的度指与顶点 V 相连的边的数目

在有向图中，以该顶点为终点的边的数目称为入度，以该顶点为起点的边的数目称为出度。度指该顶点的入的与出度之和



特别地，若d(v) =|V| - 1，则称点v为支配点，

度

Codeforces 229C Triangles

给定 n 个点的完全图，Alice 获得其中给定的 m 条边，Bob 获得剩余的边。求两个图中三角形的数量之和。三角形指长度为 3 的环。$1 \leq n \leq 10^6, 0 \leq m \leq 10^6$

度

Codeforces 229C Triangles

给定 n 个点的完全图，Alice 获得其中给定的 m 条边，Bob 获得剩余的边。求两个图中三角形的数量之和。三角形指长度为 3 的环。$1 \le n \le 10^6, 0 \le m \le 10^6$

原图中有 C_n^3 个三角形，考虑被破坏的三角形个数

被破坏的三角形由 Alice 边和 Bob 边组成

一个点在两个图中度数分别为 d_i 和 $n-1-d_i$

被破坏的三元环有 $\sum_{i=1}^{n} \frac{d_i \cdot (n-1-d_i)}{2}$ 个

子图

对一张图$G = (V, E)$，若存在另一张图$H = (V', E')$满足$V' \subseteq V$ 且 $E' \subseteq E$，则称H是G的子图$(subgraph)$，记作 $H \subseteq G$

若对 $H \subseteq G$，满足 $\forall u, v \in V'$，只要 $(u, v) \in E$，均有 $(u, v) \in E'$，则称H是G的 导出子图/诱导子图 $(induced\ subgraph)$。翻译过来就是，对于子图来说，H中点集和边集都是G的子集，而在导出子图中，仅有点集是G的子集，也就是所有与被选出来的点V相连的边都要被加入到导出子图中

若 $H \subseteq G$ 满足 $V' = V$，则称H为G的生成子图/支撑子图

显然，G是自身的子图，支撑子图，导出子图；无边图是G的支撑子图。原图G和无边图都是G的平凡子图

连通性

无向图的连通性由并查集易求（并查集等下讲），这里我们基本只讨论有向图的连通性

对于一张有向图 $G = (V, E)$，对于$u, v \in V$，若存在一条途径使得 $v_0 = u, v_k = v$，则称u可达v。由定义，任意一个顶点可达自身，任意一条边的起点可达终点。（无向图中的连通也可以视作双向可达。）

若一张有向图的节点两两互相可达，则称这张图是**强连通的(后面可能要考)**

若一张有向图的边替换为无向边后可以得到一张连通图，则称原来这张有向图是 **弱连通的**

独立集与团

- 对于图 $G=(V, E)$，若 $V' \in V$ 且 V'中任意两点都不相邻，则V'是图G的一个独立集

图 G 最大的独立集的大小记作 $\alpha(G)$。求一张图的最大独立集是NP困难的。但是，若G为一颗树，求其最大独立集是容易的，甚至可以求出当独立集大小为k时的方案数(可以思考一下如何做)

- 对于图 $G=(V, E)$，若 $V' \subseteq V$且V'中任意两个不同的顶点都相邻，则V'是图G的一个团$(clique)$。团的导出子图是完全图。如果一个团在加入任何一个顶点后都不再是一个团，则这个团是一个 **极大团**

- 一张图的最大团的大小记作$\omega(G)$，最大团的大小等于其补图最大独立集的大小

对于补图反图等定义在离散数学中应有提及，在此不多做赘述

欧拉路的判定

如果图 G 中的一个路径包括每个边恰好一次，则该路径称为欧拉路径 (Euler path)

如果一个回路是欧拉路径，则称为欧拉回路 (Euler circuit)。

具有欧拉回路的图称为欧拉图 (简称 E 图)。具有欧拉路径但不具有欧拉回路的图称为半欧拉图。

欧拉路的判定

如果图 G 中的一个路径包括每个边恰好一次，则该路径称为欧拉路径 (Euler path)

如果一个回路是欧拉路径，则称为欧拉回路 (Euler circuit)。

具有欧拉回路的图称为欧拉图 (简称 E 图)。具有欧拉路径但不具有欧拉回路的图称为半欧拉图。

要保证图连通

存在欧拉回路的充要条件：度数为奇数的点只能有 0 个

存在欧拉路径的充要条件：度数为奇数的点只能有 0 或 2 个

欧拉路的判定

如果图 G 中的一个路径包括每个边恰好一次，则该路径称为欧拉路径 (Euler path)

如果一个回路是欧拉路径，则称为欧拉回路 (Euler circuit)。

具有欧拉回路的图称为欧拉图 (简称 E 图)。具有欧拉路径但不具有欧拉回路的图称为半欧拉图。

要保证图连通

存在欧拉回路的充要条件：度数为奇数的点只能有 0 个

存在欧拉路径的充要条件：度数为奇数的点只能有 0 或 2 个

要保证图连通

存在欧拉回路的充要条件：所有点入度等于出度

存在欧拉路径的充要条件：要么所有点的出度均等与入度；要么一个点满足出度比入度多 1 (起点)，一个点满足入度比出度多 1 (终点)，其余所有点入度等于出度

有向图求欧拉路径（回路）

首先判断这张图是不是半欧拉图（欧拉图）：要求每个点的出度入度满足条件，如果这张图数半欧拉图，要确定起点必须是出度恰好大于入度的那个点。

再暴力dfs搜索这张图就能求出来了

注意如果求的是字典序最小的路径，那么在dfs之前，要对每个点的所有出点排序。

```
int n,m;
int in[N],stc[N],it[N];
int s,tp;
vector<int>g[N];
void dfs(int u){
    for(int &i=it[u];i<g[u].size();)dfs(g[u][i++]);
    stc[++tp]=u;
}
signed main(){
    ios::sync_with_stdio(false);cin.tie(0);cout.tie(0);
    cin>>n>>m;
    for(int i=1;i<=m;i++){
        int x,y;
        cin>>x>>y;
        g[x].push_back(y);
        in[y]++;
    }
    for(int i=1;i<=n;i++){
        if(abs((int)g[i].size()-in[i])>1){
            cout<<"No\n";
            return 0;
        }
        if(g[i].size()>in[i]){
            if(s){
                cout<<"No\n";
                return 0;
            }
            else s=i;
        }
    }
    dfs(s?s:1);
    if(tp!=m+1)cout<<"No\n";
    else{
        reverse(stc+1,stc+tp+1);
        for(int i=1;i<=tp;i++)cout<<stc[i]<<' ';
        cout<<'\n';
    }
}
```

无向图求欧拉路径（回路）

和有向图的方法类似的判断和dfs。

但是要注意，因为是无向图，所以当我们走过一条边的时候我们要把这条边的反向边给标记，以后就不能走了。

因为我们要做标记边的操作，所以这里最好用的建图方法是链式前向星。

```
void dfs(int u){
    for(int &i=it[u];i;){
        int j=i;
        i=e[i].next;
        if(vis[j]||vis[j^1])continue;
        vis[j]=vis[j^1]=1;
        dfs(e[j].to);
    }
    stc[++tp]=u;
}
signed main(){
    cin>>n>>m;
    for(int i=1;i<=m;i++){
        int x,y;cin>>x>>y;
        link(x,y);
        link(y,x);
        deg[x]++;
        deg[y]++;
    }
    for(int i=1;i<=n;i++)it[i]=head[i];
    for(int i=1;i<=n;i++){
        if(deg[i]%2){
            if(!s)s=i;
            else if(!t)t=i;
            else{
                cout<<"NO\n";
                exit(0);
            }
        }
    }
    dfs(s?s:1);
    if(tp!=m+1)cout<<"NO\n";
    else{
        reverse(stc+1,stc+tp+1);
        for(int i=1;i<=tp;i++)cout<<stc[i]<<' ';
        cout<<'\n';
```

并查集

并查集是一种用于管理元素所属集合的数据结构，实现为一个森林，其中每棵树表示一个集合，树中的节点表示对应集合中的元素。

顾名思义，并查集支持两种操作：
合并（Unite）：合并两个元素所属集合（合并对应的树）.
查询（Find）：查询某个元素所属集合（查询对应的树的根节点），这可以用于判断两个元素是否属于同一集合.

并查集

初始化

初始时，每个元素都位于一个单独的集合，表示为一棵只有根节点的树．方便起见，我们将根节点的父亲设为自己．

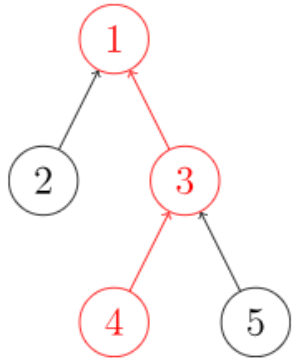
```
int fa[N];
cin>>n;
for (int i=1;i<=n;i++)
    fa[i]=i;
```

并查集

查询

我们需要沿着树向上移动，直至找到根节点

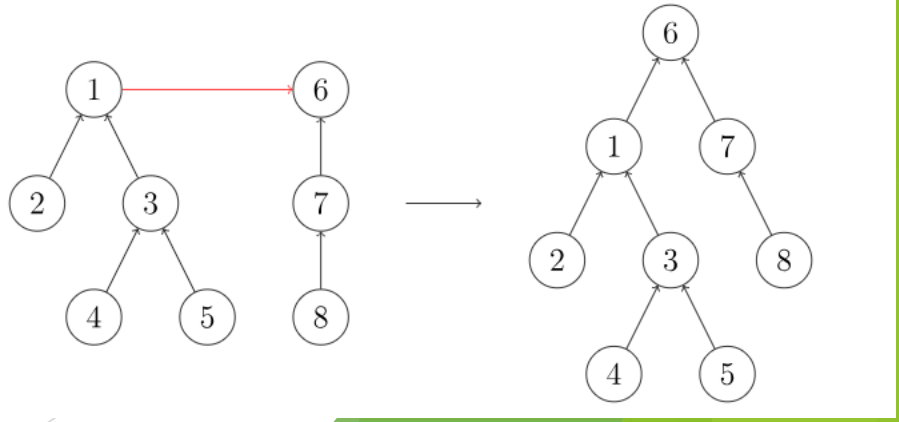
```
int find(int x)
{
    return f[x]==x?x:find(f[x]);
}
```



合并

要合并两棵树，我们只需要将一棵树的根节点连到另一棵树的根节点.

```
void unie(int x,int y)
{
    f[find(x)]=find(y);
}
```

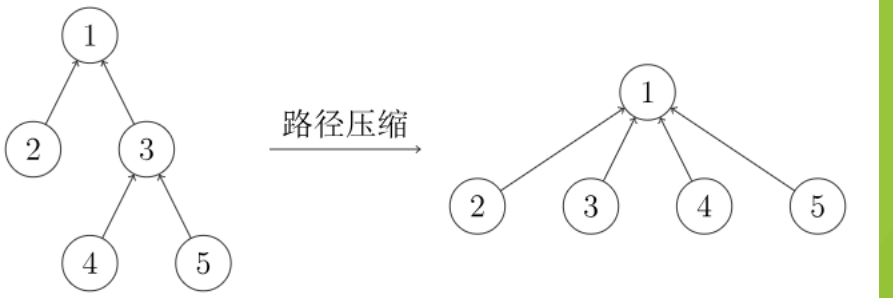


并查集

路径压缩

查询过程中经过的每个元素都属于该集合，我们可以将其直接连到根节点以加快后续查询

```
int find(int x)
{
    return f[x]==x?x:f[x]=find(f[x]);
}
```



启发式合并

合并时，选择哪棵树的根节点作为新树的根节点会影响未来操作的复杂度．我们可以将节点较少或深度较小的树连到另一棵，以免发生退化

```
void unie(int x,int y)
{
    x=find(x);
    y=find(y);
    if (x==y) return ;
    if (size[x]<size[y]) swap(x,y);
    f[x]=y;
    size[x]+=size[y];
}
```

并查集

Luogu P2024 [NOI2001] 食物链

动物王国中有三类动物 A, B, C，这三类动物的食物链构成了有趣的环形。A 吃 B，B 吃 C，C 吃 A。

现有 N 个动物，以 $1 \sim N$ 编号。每个动物都是 A, B, C 中的一种，但是我们并不知道它到底是哪一种。

有人用两种说法对这 N 个动物所构成的食物链关系进行描述：

- 第一种说法是 `1 X Y`，表示 X 和 Y 是同类。
- 第二种说法是 `2 X Y`，表示 X 吃 Y。

此人对 N 个动物，用上述两种说法，一句接一句地说出 K 句话，这 K 句话有的是真的，有的是假的。当一句话满足下列三条之一时，这句话就是假话，否则就是真话。

- 当前的话与前面的某些真的话冲突，就是假话；
- 当前的话中 X 或 Y 比 N 大，就是假话；
- 当前的话表示 X 吃 X，就是假话。

你的任务是根据给定的 N 和 K 句话，输出假话的总数。

$1 \leq N \leq 5 \times 10^4$，$1 \leq K \leq 10^5$。

100 7 1 101 1 2 1 2 2 2 3 2 3 3 1 1 3 2 3 1 1 5 5	3

并查集

种类并查集/扩展域并查集

种类并查集（Type Union-Find）是并查集的一个变种，它通过扩大原有的并查集空间来维护多种不同的关系。在处理某些问题时，比如需要区分元素之间的“朋友”和“敌人”关系，仅使用普通的并查集是不够的，因为它只能维护元素之间的一种关系。

为了解决这个问题，种类并查集将每个元素的表示扩大到2倍。这样做的目的是为了能够同时维护“朋友”关系和“敌人”关系。具体来说，如果有n个元素，我们会创建一个2*n大小的数组，其中前n个位置用来表示元素之间的“朋友”关系，而后n个位置用来表示“敌人”关系。

种类并查集和普通并查集解决问题差异

1、普通并查集，维护的是具有连通性、传递性的关系

2、但有时候，我们却需要用到种类并查集来维护另一种关系：敌人的敌人是朋友

并查集

扩展域并查集：把三类动物划分成三个域，

同类域 (1~n)、捕食域 (n+1~2n)、天敌域 (2n+1~3n)。

把 x 放入同类域，x+n 放入其捕食域，x+2n 放入其天敌域。

给 x 在其他集合内安插两个“虚拟代表”，从而实现关系传递。

x 吃 y，则 x 与 y 的天敌代表 y+2n 是同类，合并 (x, y+2n)；

x 吃 y，则 x 的捕食代表 x+n 与 y 是同类，合并 (x+n, y)；

x 吃 y，则 x 的天敌代表 x+2n 与 y 的捕食代表 y+n 是同类，

合并 (x+2n, y+n) 。

例如，n=10, 1吃2, 2吃3, 3吃4。

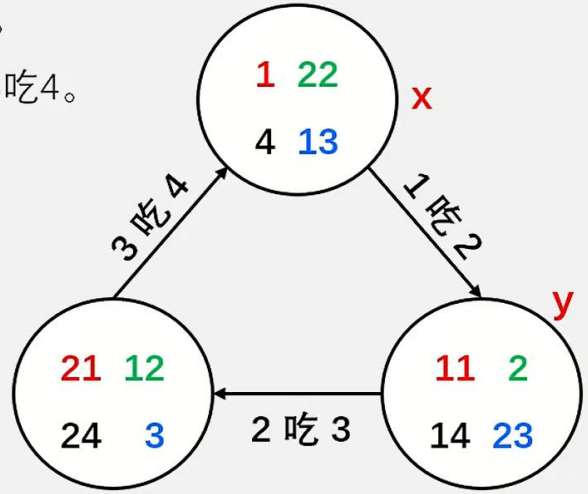
1吃2: (1,22) (11,2) (21,12)

2吃3: (2,23) (12,3) (22,13)

3吃4: (3,24) (13,4) (23,14)

通过代表 22和13，

把 1与4 合并到一起。




```
int find(int x){
  return p[x]==x?x:p[x]=find(p[x]);
}
void unset(int x,int y){
  p[find(y)]=find(x);
}
int main(){
  scanf("%d%d",&n,&k);
  for(int i=1; i<=3*n; i++) p[i]=i;
  for(int i=1; i<=k; i++){
    scanf("%d%d%d",&op,&x,&y);
    if(x>n||y>n) ans++;
    else if(op==1){ //x,y是同类
      //如果x在y的捕食域,或x在y的天敌域,是假话ans+1
      if(find(x)==find(y+n)||find(x)==find(y+n+n))ans++;
      else{   //是真话,合并同类集合
        unset(x,y);
        unset(x+n,y+n);
        unset(x+n+n,y+n+n);
      }
    }
    else{ //x吃y
      //如果他们在同类域,或者x在y的捕食域,是假话ans+1
      if(find(x)==find(y)||find(x)==find(y+n))ans++;
      else{   //是真话,合并同类集合
        unset(x,y+n+n);
        unset(x+n,y);
        unset(x+n+n,y+n);
      }
    }
  }
  printf("%d\n",ans);
}
```

并查集

带权并查集

普通的并查集仅仅记录的是集合的关系，这个关系无非是同属一个集合或者是不在一个集合。
带权并查集不仅记录集合的关系，还记录着集合内元素的关系或者说是集合内元素连接线的权值。

带权并查集的实现，需要在普通并查集的基础上，注意以下问题：
1、每个节点都记录的是与根节点之间的权值，那么在Find的路径压缩过程中，权值也应该做相应的更新，因为在路径压缩之前，每个节点都是与其父节点链接着，那个Value自然也是与其父节点之间的权值。
2、在两个并查集做合并的时候，权值也要做相应的更新，因为两个并查集的根节点不同。

并查集

带权并查集：维护元素的连通性和传递性。

把动物划分到**并查集的树**的不同的**图层**，用 d[x] 表示 x 到根的距离（权值）。

根据图层距离之差 (d[x]-d[y]+3) 模 3 的余数判断关系：

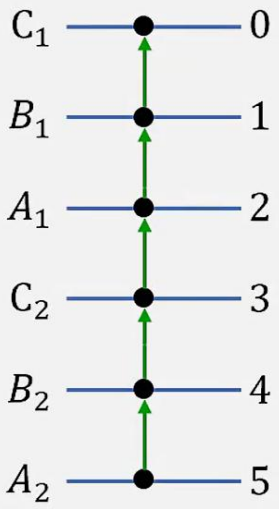
同余为 0 表示 y 是 x 的同类，

同余为 1 表示 y 是 x 的猎物，

同余为 2 表示 y 是 x 的天敌。

如图，自底向上，A_2 吃 B_2 吃 C_2 吃 A_1 吃 B_1 吃 C_1，

1. A_1 是 A_2 的同类，因 $(5-2+3)\%\,3=0$，
   同理 B_1 是 B_2 的同类，C_1 是 C_2 的同类。
2. B_1 是 A_1 的猎物，因 $(2-1+3)\,\%\,3=1$，
   B_2 也是 A_1 的猎物，因 $(2-4+3)\,\%\,3=1$。
3. C_2 是 A_1 的天敌，因 $(2-3+3)\,\%\,3=2$，
   C_1 也是 A_1 的天敌，因 $(2-0+3)\,\%\,3=2$。



并查集

- 查找时进行**边权转移**和**路径压缩**

路径压缩就是在搜索的时候找到最远的祖先，将父节点赋值。

边权转移就是在返回的时候自顶向下依次累加，维护到新根的距离。

如图，路径压缩前：

p[4]=3，p[3]=2，p[2]=1，p[1]=1；

d[4]=a，d[3]=b，d[2]=c，d[1]=0。

向上搜索，返回时，依次更新：

回到 2：t=1, p[2]=1, d[2]=d[2]+d[1]=c，p[2]=1；

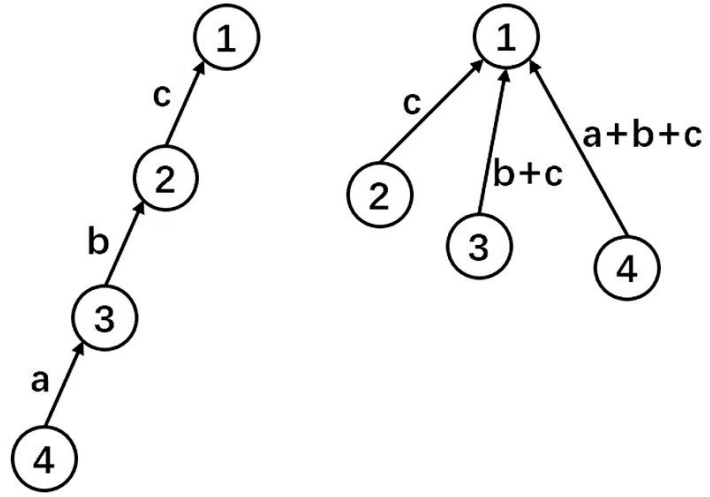
回到 3：t=1, p[3]=2, d[3]=d[3]+d[2]=b+c，p[3]=1；

回到 4：t=1, p[4]=3, d[4]=d[4]+d[3]=a+b+c，p[4]=1；

维护 x 到根的距离，即维护 x 所在的图层（x 所属的种类）

```
int p[N],d[N]; //d[x]表示x到父节点的距离

int find(int x){
  if(p[x]==x) return x;
  int t=find(p[x]);  //记录新根t
  d[x]=d[x]+d[p[x]]; //先更新x到新根t的距离
  return p[x]=t;     //再更新p[x]，路径压缩
}
```



并查集

- 合并时要维护 x 与 y 的**图层关系**

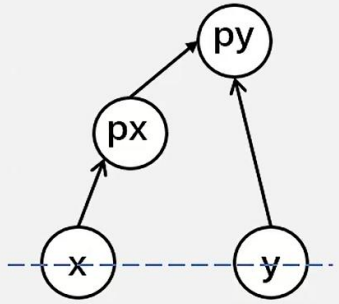
合并集合就是一棵树认另一棵树做父亲，把树根相连。

如图，把树根 px 指向 py，即 px 认 py 做父节点。

但是 px 到 py 的边权不能直接赋值为 1，因为这样就会破坏 x 与 y 的图层关系，应根据 x与y 的关系确定边权。

1. 如果 x,y 是同类，那么 x,y 应该在同一层，即 x,y 到根 py 的距离应相等，$d[px]+d[x]=d[y]$

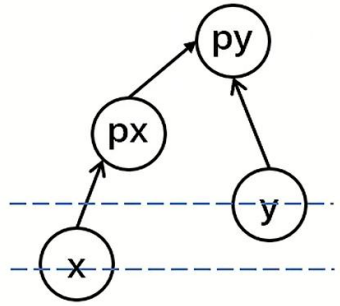
   所以 $d[px]=d[y]-d[x]$



2. 如果 x 吃 y，那么 x 应该在 y 的下一层，即 x 到根 py 的距离应多 1，$d[px]+d[x]=d[y]+1$

   所以 $d[px]=d[y]+1-d[x]$

```
else if(k==1){ //x,y是同类情况
  //x,y在同一颗树且不在同一层，则不是同类，是假话
  if(px==py&&((d[x]-d[y])%3+3)%3!=0) ans++;
  //x,y不在同一颗树，则合并集合
  if(px!=py){
    p[px]=py; //px指向py
    d[px]=d[y]-d[x];
    //x,y应在同一层，到根的距离应相等，d[px]+d[x]=d[y]
  }
}
else{ //x吃y情况
  //x,y在同一颗树且x不在y的下一层，则不满足x吃y，是假话
  if(px==py&&((d[x]-d[y])%3+3)%3!=1) ans++;
  //x,y不在同一颗树，则合并集合
  if(px!=py){
    p[px]=py; //px指向py
    d[px]=d[y]+1-d[x];
    //x应在y的下一层，到根的距离应多1，d[px]+d[x]=d[y]+1
  }
}
```



最小生成树

我们定义无向连通图的 最小生成树（Minimum Spanning Tree，MST）为边权和最小的生成树.

注意：只有连通图才有生成树，而对于非连通图，只存在生成森林.

介绍最常用的两个算法，Kruskal 算法/Prim 算法

最小生成树

Kruskal 算法

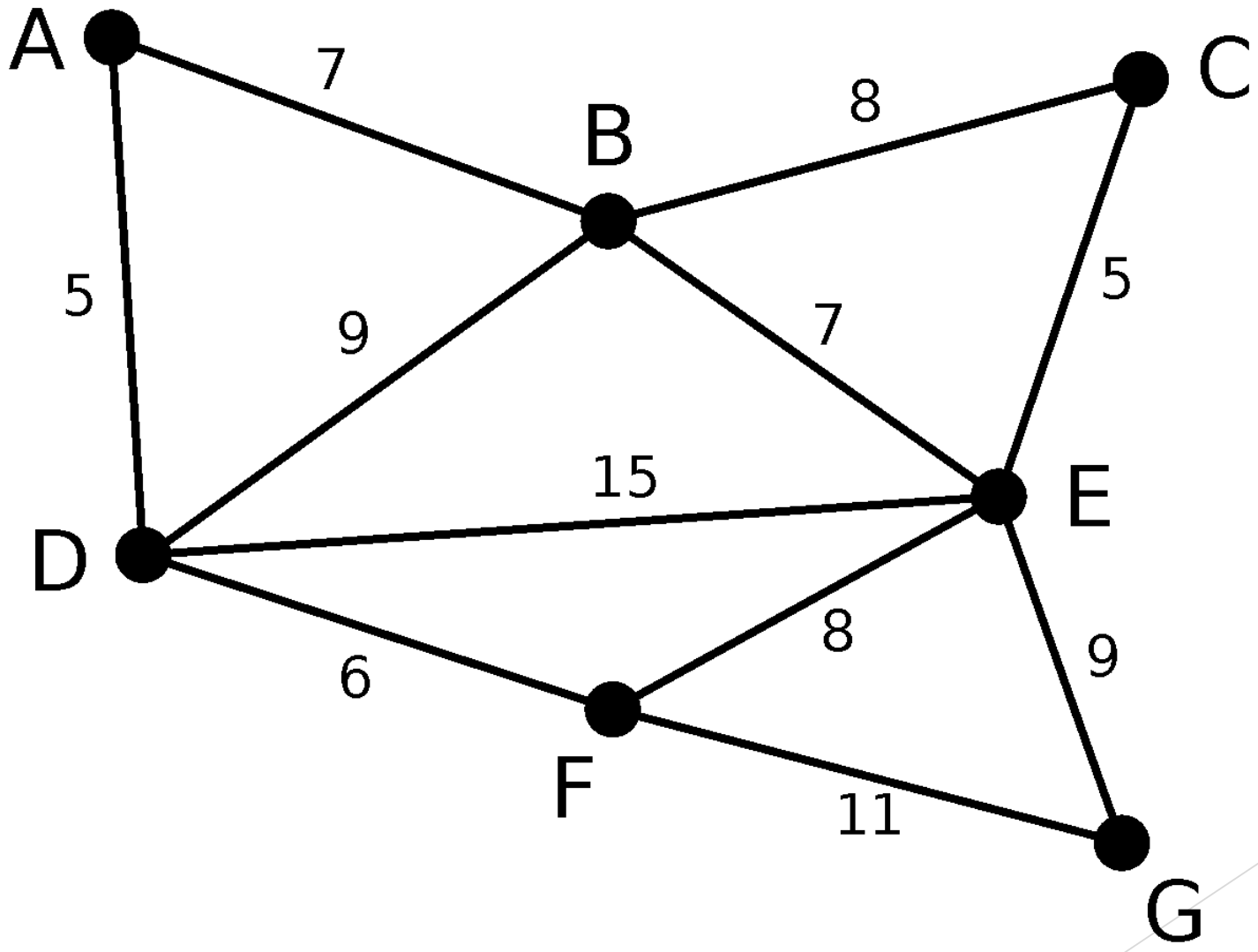
Kruskal 算法是一种常见并且好写的最小生成树算法，由 Kruskal 发明．该算法的基本思想是从小到大加入边，是个贪心算法．

具体来说，维护一个森林，查询两个结点是否在同一棵树中，连接两棵树．抽象一点地说，维护一堆集合，查询两个元素是否属于同一集合，合并两个集合．

其中，查询两点是否连通和连接两点可以使用并查集维护．

如果使用 $O(m\log m)$的排序算法，并且使用 $O(m\alpha(m,n))$ 或 $O(m\log n)$的并查集，就可以得到时间复杂度为 $O(m\log m)$ 的 Kruskal 算法．

最小生成树



最小生成树

```
int find(int x)
{
    return fa[x]==x?x:fa[x]=find(fa[x]);
}
void kruskal()
{
    sort(edge,edge+m,cmp);
    //将边的权值排序
    for(int i=0;i<m;i++) {
        eu=find(edge[i].u), ev=find(edge[i].v);
        if(eu==ev) continue;
        //若出现两个点已经联通了，则说明这一条边不需要了
        ans+=edge[i].w;
        //将此边权计入答案
        fa[ev]=eu;
        //将eu、ev合并
        if(++cnt==n-1)break;
        //循环结束条件，及边数为点数减一时
    }
}
```

最小生成树

Prim 算法

Prim 算法是另一种常见并且好写的最小生成树算法. 该算法的基本思想是从一个结点开始，不断加点（而不是 Kruskal 算法的加边）

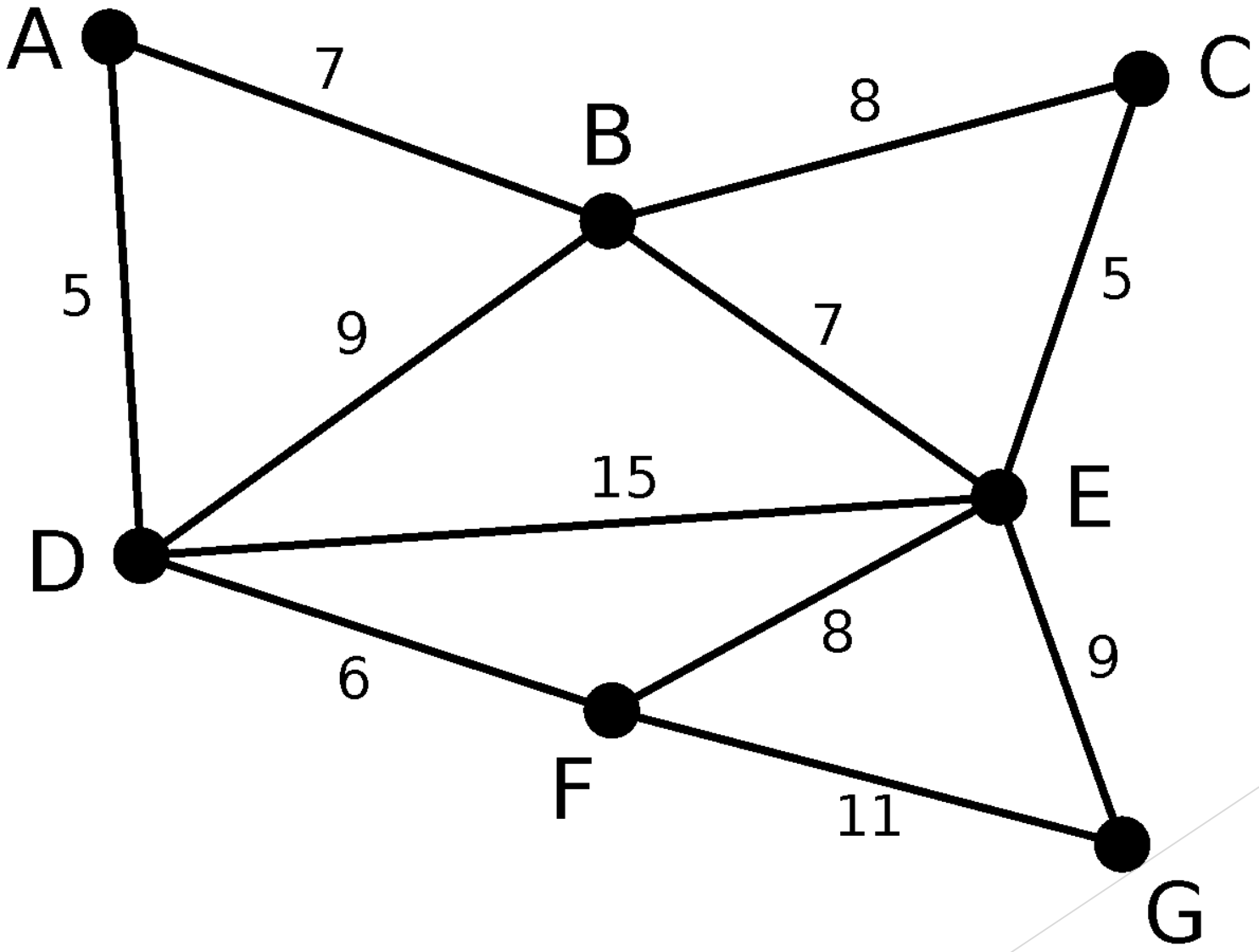
具体来说，每次要选择距离最小的一个结点，以及用新的边更新其他结点的距离.

其实跟等会会讲的 Dijkstra 算法一样，每次找到距离最小的一个点，可以暴力找也可以用堆维护.

一般情况下都使用 Kruskal 算法，在稠密图尤其是完全图上，暴力 Prim 的复杂度比 Kruskal 优，但不一定实际跑得更快.

暴力：$O(n\hat{}2 + m)$
堆优化：$O((n+m)\log n)$

最小生成树



最小生成树

```
int dis[N], vis[N];
int res = 0, cnt = 0;
priority_queue<S> q;
void Prim()
{
    memset(dis, 0x3f, sizeof(dis));
    dis[1] = 0;
    q.push({1, 0});
    while (!q.empty()) {
        if (cnt >= n) break;
        int u = q.top().u, d = q.top().d;
        q.pop();
        if (vis[u]) continue;
        vis[u] = 1;
        ++cnt;
        res += d;
        for (int i = h[u]; i; i = e[i].x) {
            int v = e[i].v, w = e[i].w;
            if (w < dis[v]) {
                dis[v] = w;
                q.push({v, w});
            }
        }
    }
}
```

拓扑排序

一个工程常被分为多个小的子工程，这些子工程被称为活动(Activity)，在有向图中若以顶点表示活动，有向边表示活动之间的先后关系，这样的图简称为 AOV 网。在 AOV 网中为了更好地完成工程，必须满足活动之间先后关系，需要将各活动排一个先后次序即为拓扑排序

拓扑排序

1. 从有向图中选取一个没有前驱的顶点，并输出之;

2. 从有向图中删去此顶点以及所有以它为尾的弧;

重复上述两步，直至图空，或者图不空但找不到无前驱的顶点为止。没有前驱--入度为零，删除顶点及以它为尾的弧--弧头顶点的入度减 1。

可能有多解

无环图所有点都可排序，有环图会剩余点无法排序

拓扑排序

```
cin>>n>>m;
for(int i=1;i<=m;i++){
    int x,y;cin>>x>>y;
    g[x].push_back(y);
}

queue<int>q,ans;
for(int i=1;i<=n;i++)if(deg[i]==0)q.push(i);

while(q.size()){
    int u=q.front();q.pop();
    ans.push_back(u);//推入答案
    for(auto v:g[u]){
        deg[v]--;
        if(deg[v]==0)q.push(v);//如果度数为0，说明没有点连向它
    }
}
```

拓扑排序

HDU1285 确定比赛名次

有 n 个队伍进行了 m 场比赛，给定 m 场比赛的胜负状况，让你对所有队伍进行排序。保证有唯一合法解。$1 \leq n \leq 500$

输入数据：　　目前排名：　　队列：1

6 6

1 2

1 3

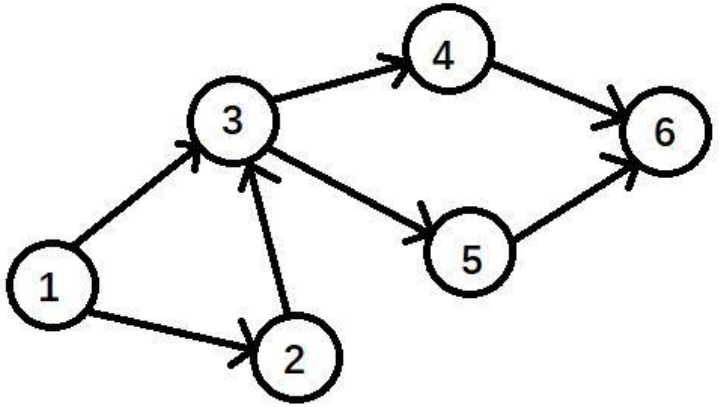
2 3

3 5

3 4

4 6

5 6



拓扑排序

HDU1285 确定比赛名次

有 n 个队伍进行了 m 场比赛，给定 m 场比赛的胜负状况，让你对所有队伍进行排序。保证有唯一合法解。$1 \leq n \leq 500$

输入数据：　　目前排名：1　　队列：2

6 6

1 2

1 3

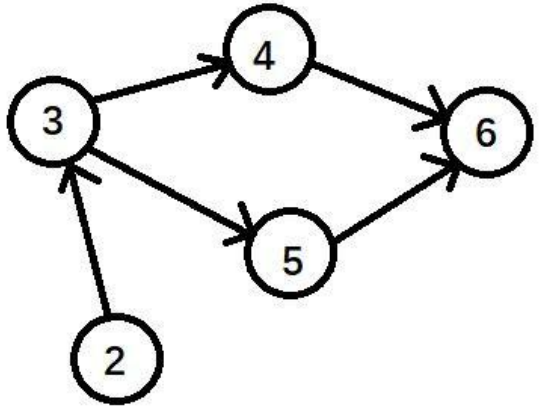
2 3

3 5

3 4

4 6

5 6



拓扑排序

HDU1285 确定比赛名次

有 n 个队伍进行了 m 场比赛，给定 m 场比赛的胜负状况，让你对所有队伍进行排序。保证有唯一合法解。$1 \leq n \leq 500$

输入数据：　　目前排名：1 2　　队列：3

6 6

1 2

1 3

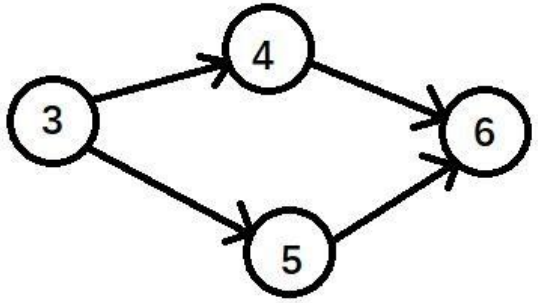
2 3

3 5

3 4

4 6

5 6



拓扑排序

HDU1285 确定比赛名次

有 n 个队伍进行了 m 场比赛，给定 m 场比赛的胜负状况，让你对所有队伍进行排序。保证有唯一合法解。$1 \leq n \leq 500$

输入数据：　　目前排名：1 2 3　　队列：5 4

6 6

1 2

1 3

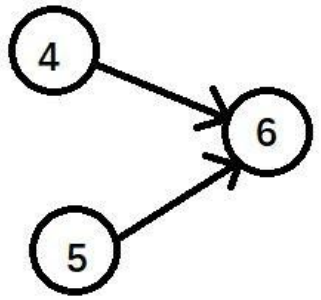
2 3

3 5

3 4

4 6

5 6



拓扑排序

HDU1285 确定比赛名次

有 n 个队伍进行了 m 场比赛，给定 m 场比赛的胜负状况，让你对所有队伍进行排序。保证有唯一合法解。$1 \leq n \leq 500$

输入数据：　　目前排名：1 2 3 5　　队列：4

6 6

1 2

1 3

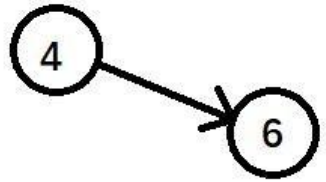
2 3

3 5

3 4

4 6

5 6



拓扑排序

HDU1285 确定比赛名次

有 n 个队伍进行了 m 场比赛，给定 m 场比赛的胜负状况，让你对所有队伍进行排序。保证有唯一合法解。$1 \leq n \leq 500$

输入数据：　　目前排名：1 2 3 5 4　　队列：6

6 6

1 2

1 3

2 3

3 5

3 4

4 6

5 6



拓扑排序

HDU1285 确定比赛名次

有 n 个队伍进行了 m 场比赛，给定 m 场比赛的胜负状况，让你对所有队伍进行排序。保证有唯一合法解。$1 \leq n \leq 500$

输入数据：

6 6

1 2

1 3

2 3

3 5

3 4

4 6

5 6

目前排名：1 2 3 5 4 6

题目要求唯一合法解，字典序最小的拓扑序列

用普通的拓扑排序求出来的序列不唯一

现在求出来的1 2 3 5 4 6并不是字典序最小

所以把队列换成优先队列就可以达到最小字典序的目的

拓扑排序

Codeforces 510C Fox And Names

给 n 个字符串，每个长度最多为 s。让你输出一个自定义的字母表使得给定的字符串为字典序升序。$1 \leq n \leq 100, 1 \leq s \leq 100$

拓扑排序

Codeforces 510C Fox And Names

给 n 个字符串，每个长度最多为 s。让你输出一个自定义的字母表使得给定的字符串为字典序升序。$1 \leq n \leq 100, 1 \leq s \leq 100$

每两个字符串进行逐个比较，找到两个字符串第一个不同的位置的字母，例如：aba和aaa，这两个不停地第一个位置是b和a，假设aba在aaa前面，那么说明在字母表中，b在a前面，连一条b到a的边。建完有向图之后，拓扑排序，如果出现了环（拓扑序数组大小不等于26）说明排不出来。

如果有的字母没有出现题目没有最小字典序之类的要求也无所谓，因为题目没要求

拓扑排序

Problem - J - Codeforces

The annual CCPC (*Cipher & Code Penetrating Competition*) is approaching, and this year's competition has a unique way of releasing the problems.

This competition has a total of n problems, numbered from 1 to n. However, unlike previous years, this year's problems are not all unlocked at the beginning of the competition; instead, they are unlocked gradually. Specifically, the organizing committee has determined a directed graph with n nodes based on the relationships between the problems, where the nodes are numbered from 1 to n and node i represents the i-th problem. Initially, each problem has an energy level of 0, and each problem has a parameter a_i, which indicates that if the energy level of this problem is greater than or equal to a_i, the problem will be immediately unlocked. Once unlocked, all directed edges originating from that node will **simultaneously** transfer 1 point of energy to the corresponding destination problem, which takes w_i seconds.

However, the organizing committee has discovered that some problems may never be unlocked, which is something they want to avoid. Therefore, they designed k forced refreshers to help participants progress. Each forced refresher controls some problems and will be activated at time t_i seconds, immediately modifying the parameter a_i of the problems it controls to 0.

Now, for each problem, you want to know its earliest unlock time or determine if it can never be unlocked.

拓扑排序

题目大意

- 给你一个有向图，一个点被激活当且仅当其入边被激活的数 目大于等于给定的参数 ai，一个点被激活后会立即同时激 活其所有出边，激活一条边的需要的时间为 wj 秒
- 有一些特殊的装置，可以在开始后 x 秒将给定点集的 ai 变 成 0.
- 问你选中的某个点能否被激活，以及激活它需要的最小时间
- $n \le 1e5$, $m \le 1e6$, $k \le 1e5$, $ai \le 1e5$, wi , $xi \le 1e9$

拓扑排序

思路

- 我们考虑类似拓扑排序的思想，每次有一条边被激活后，其会对到达的点做出 1 的贡献。

- 而特殊装置可以将指定点集的 ai 变成 0，可以看作一个超级源点向其连了一条长度为 x ，贡献为 inf 的一条边。

- 利用优先队列来实现上述过程即可

特别地，在拓扑排序的过程中进行dp也是常见的手法。当一个点i入队后，说明dp[i]已经汇总了其前驱节点的所有信息，可以转移到其临接点，保证dp的无后效性。

特殊图最短路

求树上两点最短路：dfs，或者lca（树论应该讲了）

有向无环图最短路：拓扑排序

无权图（所有边都相等）：bfs

边权只有0或1：01bfs

单源最短路

从源点出发，到其它所有点的最短路径长度

定义 dist[x] 表示点 x 距离源点的临时最短路，初始化为 INF

基本操作：松弛

if(dist[v] > dist[u] + g[u][v]) dist[v] = dist[u] + g[u][v]

从源点开始不断进行松弛，点 x 不能再被松弛时，dist[x] 为 x 距离源点的最短路

Dijkstra

要求图中不存在负权边

最小的临时最短路 dist[x] 无法再被松弛，所以每次找最近的点进行松弛操作

Dijkstra

1、在开始之前，认为所有的点都没有进行过计算，dist[] 全部赋值为极大值 (dist[] 表示各点当前到源点的最短距离)

2、源点的 dist 值明显为 0

3、计算与 s 相邻的所有点的 dist 值——dist[v] = g[s][v]

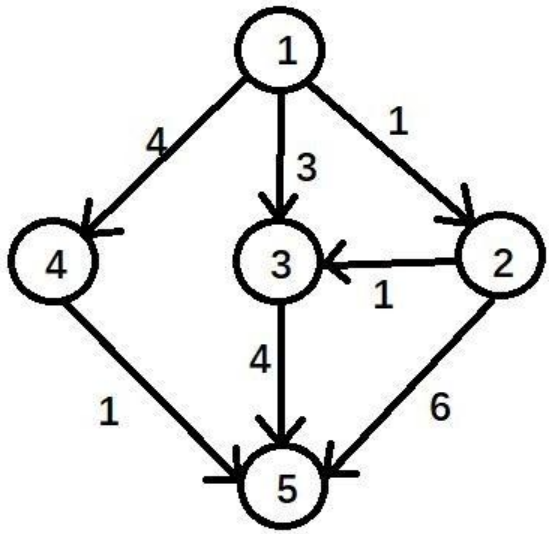
4、还没算出最短路的点中 dits[] 最小的一个点 u，其最短路就是当前的 dist[u]

5、对于与 u 相连的所有点 v，若 dist[u]+g[u][v] 比当前的 dist[v] 小，更新 dist[v]

6、重复 4,5 直到源点到所有点的最短路都已求出

Dijkstra

	1	2	3	4	5
dist	0	inf	inf	inf	inf



此时的优先队列:[0,1]

1到它自己的距离为0，肯定最小，所以把1加入优先队列，用1去松弛其他点

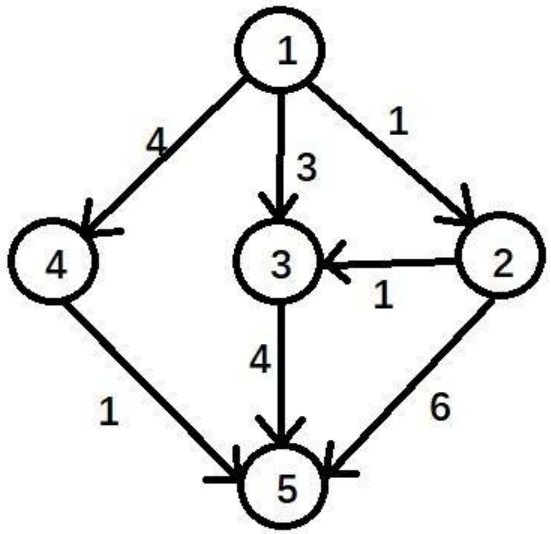
dist[1]+1<dist[2]所以dist[2]=dist[1]+1，2加入队列

dist[1]+3<dist[3]所以dist[3]=dist[1]+3，3加入队列

dist[1]+4<dist[4]所以dist[4]=dist[1]+4，4加入队列

Dijkstra

	1	2	3	4	5
dist	0	1	3	4	inf



此时的优先队列:[1,2] [3,3] [4,4]

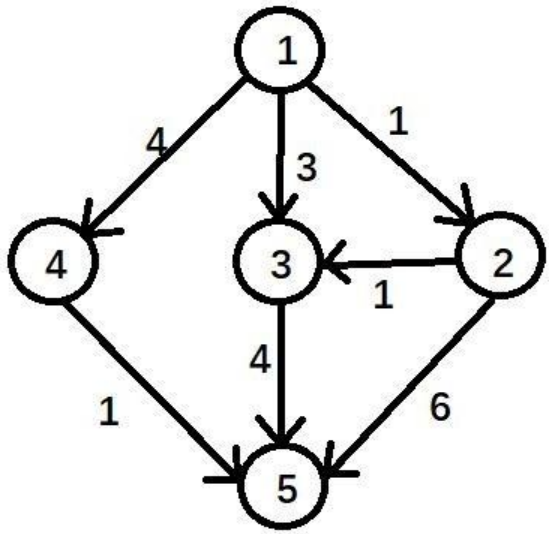
由于dist[2]是此时最小的，可以说明2不会再被松弛（因为假设2还能被某个点k松弛，那么k肯定在2之前就已经加入答案并且用来松弛2了，所以如果某个点在优先队列队首，那么他一定无法再被松弛）

dist[2]+1<dist[3]所以dist[3]=dist[2]+1，3加入队列

dist[2]+6<dist[5]所以dist[5]=dist[2]+6，5加入队列

Dijkstra

	1	2	3	4	5
dist	0	1	2	4	7



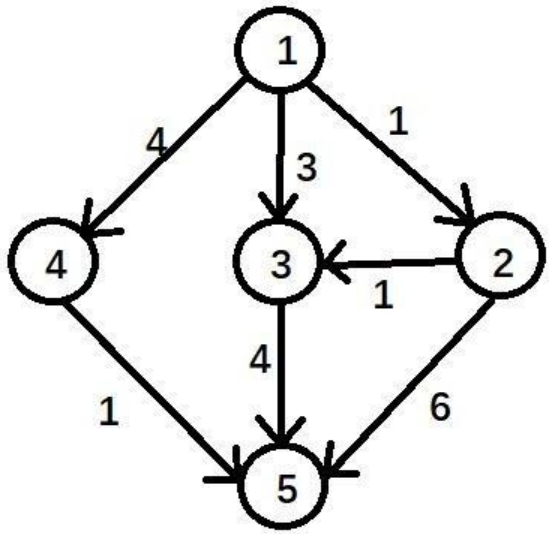
此时的优先队列:[2,3] [3,3] [4,4] [7,5]

现在优先队列队首是3，说明dist[3]是此时最小的，用3来松弛其他点

dist[3]+4<dist[5]所以dist[5]=dist[3]+4，5加入队列

Dijkstra

	1	2	3	4	5
dist	0	1	2	4	6

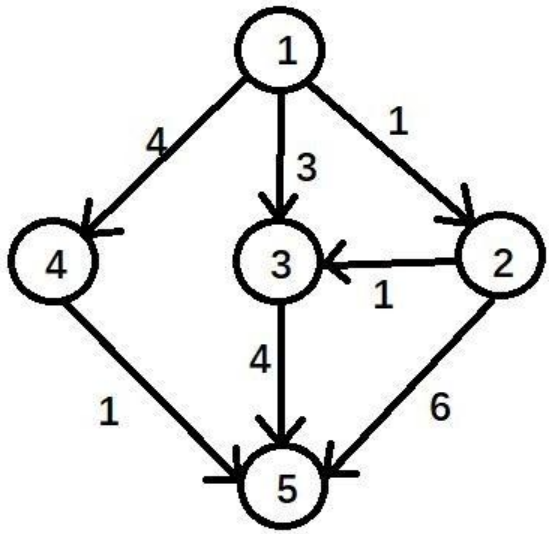


此时的优先队列:[3,3] [4,4] [6,5] [7,5]

现在队首还是3，但是dist[3]=2，队首的距离是3，说明这个队首是之前某一次其他点对3的松弛，但是并不是最优的松弛，所以不用这个距离去松弛其他点，continue就行

Dijkstra

	1	2	3	4	5
dist	0	1	2	4	7



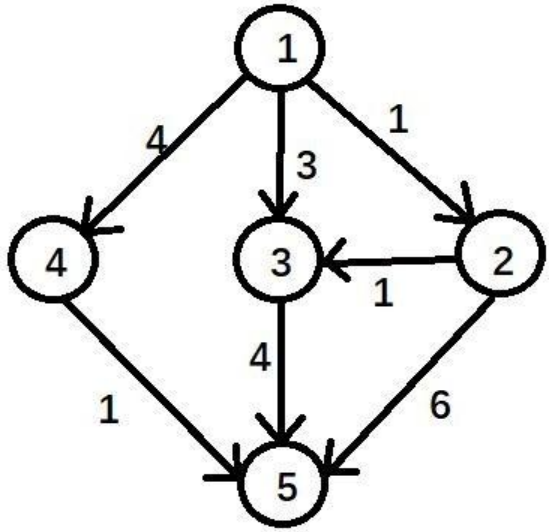
此时的优先队列:[4,4] [6,5] [7,5]

现在优先队列队首是4，说明dist[4]是此时最小的，用4来松弛其他点

dist[4]+1<dist[5]所以dist[5]=dist[4]+1，5加入队列

Dijkstra

	1	2	3	4	5
dist	0	1	2	4	5



此时的优先队列:[5,5] [6,5] [7,5]

现在优先队列队首是5，说明dist[5]是此时最小的，用5来松弛其他点

此时没有点需要被松弛了，所有点距离1号点的最短路都求出来了

优先队列那几个往后都被continue掉了（原因和上次的3一样）

Dijkstra

```cpp
typedef pair<int,int> PII;//重命名pair，写起来更方便
#define ft first
#define sd second
void dijkstra(){
    //小根堆
    priority_queue<PII,vector<PII>,greater<PII>>q;
    q.push({0,1});
    while(q.size()){
        //如果队首的距离不等于该点的dist，跳过
        if(dist[q.top().ft]!=q.top().sd)continue;
        int u=q.top().sd;q.pop();
        for(int i=head[u];i;i=nex[i]){
            int v=e[i];
            //松弛操作，如果松弛成功，就推进优先队列里
            if(dist[v]>dist[u]+w[i]){
                dist[v]=dist[u]+w[i];
                //注意这里距离在先，点序号在后
                //因为pair默认的排序是先按照第一项从小到大，再按照第二项
                q.push({dist[v],v});
            }
        }
    }
}
```

Dijkstra

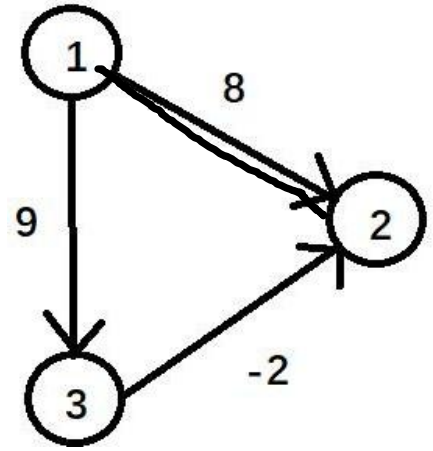
注意：

Dijkstra的核心操作是松弛，实际上，将dijkstra看成堆优化的dp也未尝不可。所以，带着一种用当前节点的信息更新相邻点的信息的思想考虑不失为一种好的选择。实际应用中，dis数组的含义可能不仅仅是距离，需要根据具体情况作出恰当的定义。

这种最短路算法所处理的是广义上的“单源”，假如有多个源点，欲求每个点到任意一个源点的最短路，同样可以使用dijkstra解决

为什么Dijkstra只能处理无负权图？

因为*dijkstra*用了贪心的思想，每次选取的是当前最短的边进行松弛，当边都是正权时，松弛后边权一定比当前最短边大，所以满足贪心的条件，有了负边后不满足贪心的条件所以不能用*dijkstra*计算带有负边的单源最短路



如图，如果按照dijkstra的思想，会得到2的距离为8，3的距离为9，因为目前dist[2]<dist[3]

所以用2去松弛其他点，而不是用3去

松弛2 所以得到2的最短距离为8

但是显而易见，dist[2]=7

SPFA

每次松弛只需要对上一轮被松弛的点的邻边进行

Shortest Path Faster Algorithm

1、在开始之前，认为所有的点都没有进行过计算，dist[] 全部赋值为极大值 (dist[] 表示各点当前到源点的最短距离)

2、源点的 dist 值明显为 0

3、将源点加入队列

4、取出队头，松弛其邻边，如果邻边终点不在队列中便放入队尾

5、重复 4 直到队列空或某点入队超过 n 次

SPFA

SPFA求最短路：

```
queue<int>q;
q.push(1);vis[1]=1;
while(!q.empty()){
    int t=q.front();q.pop();vis[t]=0;
    for(int i=head[t];i!=0;i=a[i].next){
        if(dis[a[i].to]>dis[t]+a[i].val){
            dis[a[i].to]=dis[t]+a[i].val;
            if(!vis[a[i].to])q.push(a[i].to),vis[a[i].to]=1;
        }
    }
}
```

SPFA判断负环：

```
queue<int>q;
for(int i=1;i<=n;i++)q.push(i),vis[i]=1;
while(!q.empty()){
    int t=q.front();q.pop();vis[t]=0;
    for(int i=head[t];i;i=a[i].next){
        if(dis[a[i].to]>dis[t]+a[i].val){
            dis[a[i].to]=dis[t]+a[i].val;
            cnt[a[i].to]=cnt[t]+1;
            flag|=cnt[a[i].to]>=n;
            if(!vis[a[i].to])q.push(a[i].to),vis[a[i].to]=1;
        }
        if(flag)break;
    }
    if(flag)break;
}
if(flag)cout<<"Yes\n";
else cout<<"No\n";
```

最短路方法比较

BFS（01BFS）：边权为 1（边权为 01），每次扩展一层

拓扑排序 +dp：有向无环图，入度为 0 的节点不会再被更新

Dijkstra：非负权边，贪心

spfa：暴力，反复松弛

多源汇最短路

执行一次算法

得到任意两点间的最短路

复杂度 $O(n^3)$

原理基于动态规划

```
for(int k = 1; k<=n; k++)
    for(int i = 1; i<=n; i++)
        for(int j = 1; j<=n; j++)
            g[i][j] = min(g[i][k] + g[k][j] , g[i][j]);
```

多源汇最短路

HDU2544 最短路

给定 n 个点 m 条边的无向带权图，求给定两点间的最短路。

$n \leq 100, m \leq 10000$

多源汇最短路

HDU2544 最短路

给定 n 个点 m 条边的无向带权图，求给定两点间的最短路。
$n \leq 100, m \leq 10000$

EOlymp975 Floyd

给定含 n 个点的有向图的邻接矩阵，求任意两点的最短路中最长的一条。$1 \leq n \leq 100$

多源汇最短路

思考，当边权带修时，更新dis数组的代价是多少？

$$
\begin{aligned}
&1 \leq N \leq 500 \\
&0 \leq M \leq 10^5 \\
&1 \leq A_i < B_i \leq N \\
&1 \leq C_i \leq 10^9 \\
&0 \leq K \leq N \\
&1 \leq T \leq 10^9 \\
&1 \leq D_1 < \cdots < D_K \leq N \\
&1 \leq Q \leq 1000
\end{aligned}
$$

AtCoder Country has N cities numbered from 1 to N, M roads, and K airports.

The i-th road connects cities A_i and B_i bidirectionally and takes C_i hours to travel. There are airports in cities $D_1, \ldots, D_K$, and you can travel between cities with airports in T hours.

Process Q queries in order. Each query is one of the following three types:

- `1 x y t`: A road connecting cities x and y bidirectionally in t hours is built.
- `2 x`: An airport is built in city x.
- `3`: Let $f(x, y)$ be the smallest number of hours needed to reach city y from city x using roads and airports if reachable, and 0 if unreachable. Find $\sum_{x=1}^{N} \sum_{y=1}^{N} f(x, y)$.

多源汇最短路

- 考虑机场部分：
可以对所有机场建超级源点，边权为T，双向边，同时将所有边权倍长，最后答案除以2即可

- 经过上述处理，操作1和2本质上就是同一种操作->修改边权
对于修改的边权u，v，我们枚举(i,j)，修改的路径只能是i->u->v->j或者i->v->u->j，暴力更新即可

- 复杂度O($n^3 + n^2 \times q$)

扩展：次短路

给定一张N个点、M条边的无向图，每条边有边权。请求出从节点1到节点N的严格次短路的长度。

可以从两个方向思考

扩展：次短路

考虑对每个节点u维护两个距离信息：
$d1_u$: 从源点s到节点u的最短路
$d2_u$: 从源点s到节点u的次短路

当我们从优先队列中取出一个状态(d,u)，表示发现一条从s到u长度为d的路径时，对于u的每个邻居v（设边权为w），新的路径长度为d'。我们用d'来尝试更新v的状态

- if $d' < d_1[v]$: $d_2[v] \leftarrow d_1[v]$, $d_1[v] \leftarrow d'$

  更新后，将 $(d_1[v], v)$ 和 $(d_2[v], v)$ 两个状态都推入优先队列。

- else if $d_1[v] < d' < d_2[v]$: $d_2[v] \leftarrow d'$

  更新后，将新的次短路状态 $(d_2[v], v)$ 推入优先队列。

扩展：次短路

- 在执行dijkstra时，我们是可以建立一棵最短路树的（SPT）
- 类比用dfs遍历整张图时，给每个点打上dfn标记；当使用bfs遍历时，同样也可以进行这样的操作，过程中标记每个节点的父节点。此时，对应的是边权全部为1的情况
- 当扩展到边权为w时，同理可建立一棵SPT
- 显然在SPT上，从1到n的路径p1即为最短路。考虑次短路的情况，必然存在一条边为非树边

扩展：次短路

定义：
d_s(u): 从源点s到u的最短路
d_t(u): 从汇点t到u的最短路

考虑枚举图中的每一条边(u,v)，则次短路长度可以由下式精确定义：

$$d_2(t) = \min_{\substack{(u,v)\in E \\ d_s[u]+w(u,v)+d_t[v]>d_s[t]}} (d_s[u] + w(u,v) + d_t[v])$$

在理想时间限内解决k短路问题需要较高级数据结构支持，在此按下不表

浅谈图与DP

- 一般为二维，不注重像$dijkstra$那样的松弛顺序，先枚举要维护的第二维，再去枚举点实现转移

- 例：
一张带权有向图，定义$f(p)$为路径p上所有边权乘积，对于每个点，求从该点出发走j步后所走出的路径中最大的$f(p)$ $(j \leq 30, n \leq 2e5)$

浅谈图与DP

转移方程易推，注意枚举顺序

```
std::vector<std::array<int, 30>> f(n + 1);
for(int i = 1; i <= n; i++) f[i][0] = 1;
for(int j = 1; j < 30; j++){
    for(int i = 1; i <= n; i++){
        for(auto& [v, w]: adj[i]) f[i][j] = std::max(f[i][j], f[v][j - 1] * w);
    }
}
```

浅谈图与DP

You are given a simple undirected connected graph of n vertices and m edges.

There is a token in vertex 1. We consider the initial time to be at 0 seconds. After t seconds, if the token is in vertex u, you must do exactly one of the following:

- wait one second,
- move the token through the $(t \bmod \deg(u) + 1)^*$-th edge of u, which takes one second.

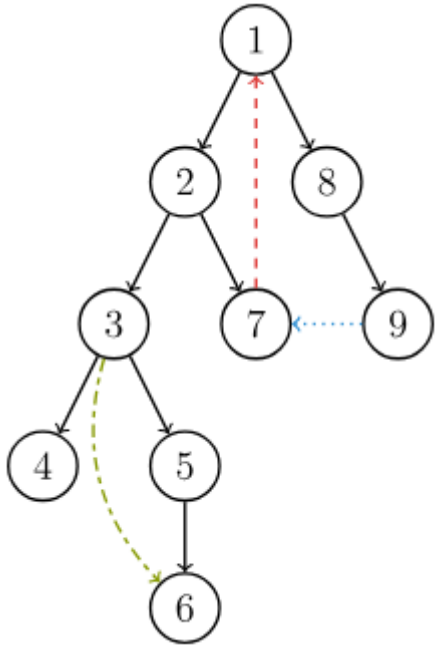
The order of edges of a vertex is the order that they appear in the input.

Calculate the minimum time required to move the token from vertex 1 to vertex n, and the minimum time spent waiting that can be achieved while minimizing the total time.

由于选择走哪条边是由时间确定的，不妨设$f_{i,j}$表示第j秒走到i点时所浪费的最小时间。若当前选择停在i点，有$f_{i,j+1} \leftarrow f_{i,j} + 1$；否则，有$f_{v,j+1} \leftarrow \min(f_{v,j+1}, f_{i,j})$，转移顺序依然是先j后i。可以用滚动数组优化空间

连通分量与Tarjan

DFS 生成树



有向图的 DFS 生成树主要有 4 种边:

1.树边：示意图中以黑色边表示，每次搜索找到一个还没有访问过的结点的时候就形成了一条树边。

2.反祖边：示意图中以红色边表示(7 ->1)，也被叫做回边，即指向祖先结点的边。

3.横叉边：示意图中以蓝色边表示（即9 ->7），它主要是在搜索的时候遇到了一个已经访问过的结点，但是这个结点并不是当前结点的祖先。

4. 前向边：示意图中以绿色边表示(3->6),它是在搜索的时候遇到子树中的结点的时候形成的。

连通分量与Tarjan

考虑 DFS 生成树与强连通分量之间的关系

如果结点u是某个强连通分量在搜索树中遇到的第一个结点，那么这个强连通分量的其余结点肯定是在搜索树中以u为根的子树中。结点u被称为这个强连通分量的根

我们在dfs时按照每个节点被访问到的顺序给图中每个节点以(1 ~ n)的整数标记，这个整数被称为时间戳，记作dfn[u]。请记住这个数组，在树中也会及经常被用到

连通分量与Tarjan

Tarjan 算法能够在线性复杂度内求得有向图的所有强连通分量。基于对图进行 DFS。我们视每个连通分量为搜索树中的一棵子树，在搜索过程中，维护一个栈，每次把搜索树中尚未处理的节点加入栈中。

为每个结点u维护了以下几个变量

1. dfn_u：深度优先搜索遍历时结点 u 被搜索的次序。
2. low_u：在 u 的子树中能够回溯到的最早的已经在栈中的结点。设以 u 为根的子树为 $Subtree_u$。low_u 定义为以下结点的 dfn 的最小值：$Subtree_u$ 中的结点；从 $Subtree_u$ 通过一条不在搜索树上的边能到达的结点。

另外用scc数组记录u属于哪个连通分量

根据dfs的搜索顺序，有如下性质：

一个结点的子树内结点的 dfn 都大于该结点的 dfn，从根开始的一条路径上的 dfn 严格递增，low 严格非降

连通分量与Tarjan

按照深度优先搜索算法搜索的次序对图中所有的结点进行搜索，维护每个结点的dfn与low变量，且让搜索到的结点入栈。每当找到一个强连通元素，就按照该元素包含结点数目让栈中元素出栈

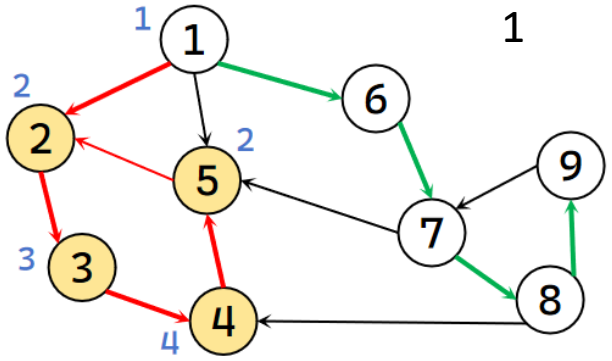
1. v未被访问：继续对v进行深度搜索。在回溯过程中，用 low_v更新low_u。因为存在从u到v的直接路径，所以v能够回溯到的已经在栈中的结点，u也一定能够回溯到

2. v被访问过，已经在栈中：根据 low 值的定义，用 dfn_v更新low_u

3. v被访问过，已不在栈中：说明v已搜索完毕，其所在连通分量已被处理，所以不用对其做操作

```cpp
void tarjan()//directed,DAG
{
    std::vector<int> dfn(n + 1),low(n + 1),bj(n + 1);
    std::vector<bool> vis(n + 1);
    std::stack<int> s;
    int cnt = 0, tot = 0;
    auto dfs = [&](this auto&& dfs,int i)->void
    {
        dfn[i] = low[i] = ++cnt;
        vis[i] = 1;
        s.emplace(i);
        for(auto& [j, w]: adj[i]) if(!dfn[j]) dfs(j),low[i] = std::min(low[i], low[j]);
        else if(vis[j]) low[i] = std::min(low[i], dfn[j]);
        if(low[i] == dfn[i])
        {
            tot++;
            while(1)
            {
                int z = s.top();
                s.pop();
                vis[z] = 0;
                bj[z] = tot;
                if(i == z) break;
            }
        }
    };//luogu may not support C++23,change it to C++20 version
    for(int i = 1; i <= n; i++) if(!dfn[i]) dfs(i);
}
```

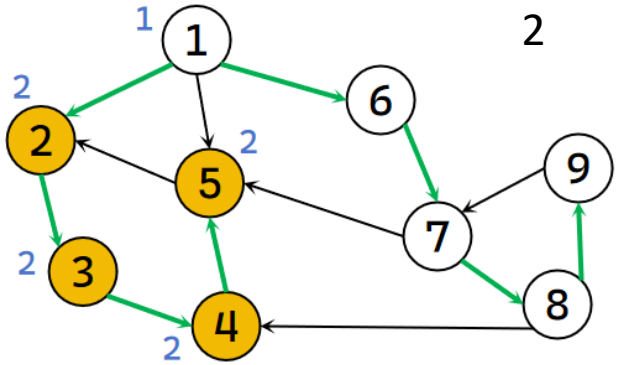
连通分量与Tarjan



s = {1,2,3,4,5}

洛谷@cold_jelly

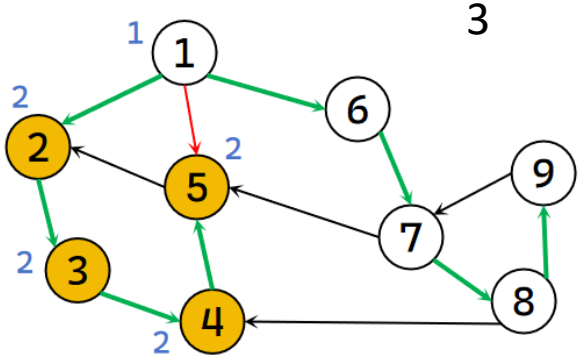
->



s = {1}

洛谷@cold_jelly

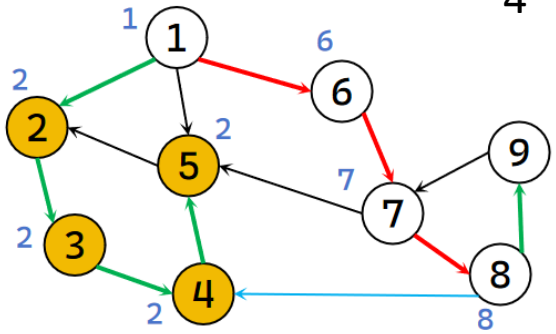
->



s = {1}

洛谷@cold_jelly

->

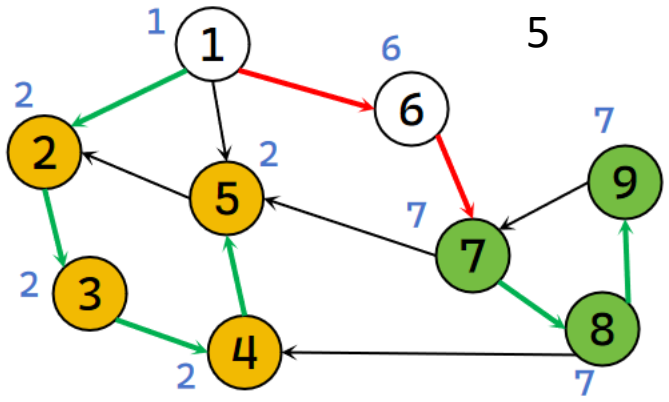


s = {1,6,7,8}

洛谷@cold_jelly

->

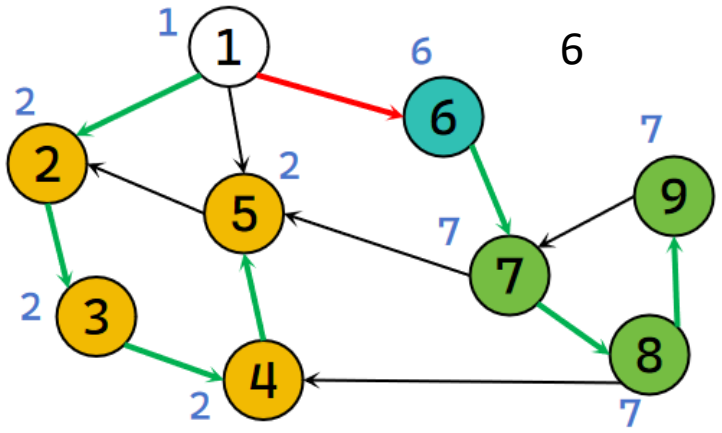
连通分量与Tarjan



s = {1,6}

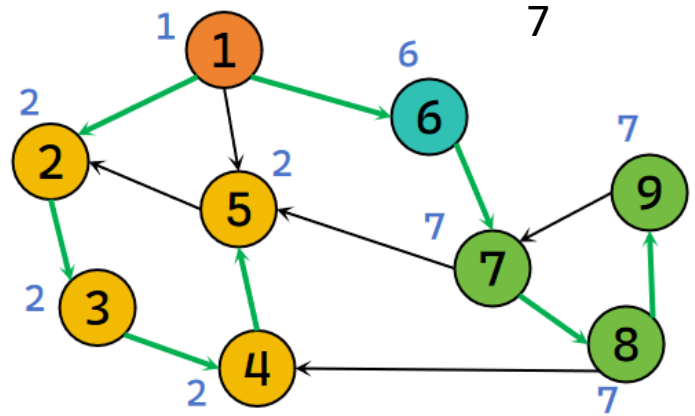
洛谷@cold_jelly

->



s = {1}

洛谷@cold_jelly



s = {}

洛谷@cold_jelly

缩点

- 我们为什么要找强连通分量？或者说，在执行完tarjan算法后，我们可以得到什么？

- 在某些题目中，一个强连通分量中的所有点都是等价的，这个时候，就可以将这些等价点看作同一个点，建立新图

- 由强连通分量的性质，在新图中是不存在环路的，否则可以再次缩成一个更大的分量。这样，原图变为一个DAG，方便我们进行dp等操作

缩点

上面我们讨论的都是有向图的情况，那么无向图如何进行缩点呢？

```
auto dfs = [&](this auto&& dfs,int i, int fa)->void
{
    dfn[i] = low[i] = ++cnt;
    vis[i] = 1;
    s.emplace(i);
    for(auto& [j, w]: adj[i]) {
        if(j == fa) continue;// difference is here
        if(!dfn[j]) dfs(j, i),low[i] = std::min(low[i], low[j]);
        else if(vis[j]) low[i] = std::min(low[i], dfn[j]);
    }
    if(low[i] == dfn[i])
    {
        tot++;
        while(1)
        {
            int z = s.top();
            s.pop();
            vis[z] = 0;
            bj[z] = tot;
            if(i == z) break;
        }
    }
```

缩点

- 区别就是在进行无向图缩点时，需要考虑前驱节点的情况
- 缩完点后建立新图，可以得到一棵树或者森林，使用树相关的算法解决问题
- 有些题目中，需要将每个点的贡献计算到缩成的新点中，参与后续的算法执行

缩点

每头奶牛都梦想成为牛棚里的明星。被所有奶牛喜欢的奶牛就是一头明星奶牛。所有奶牛都是自恋狂，每头奶牛总是喜欢自己的。奶牛之间的“喜欢”是可以传递的——如果 A 喜欢 B，B 喜欢 C，那么 A 也喜欢 C。牛栏里共有 N 头奶牛，给定一些奶牛之间的爱慕关系，请你算出有多少头奶牛可以当明星。 $1 \leq N \leq 10^4$，$1 \leq M \leq 5 \times 10^4$。

进行缩点，这样可以保证每个连通块中的奶牛全部相互喜欢。只需要找出出度为0的连通分量的大小，即该连通分量中的奶牛不会喜欢该分量以外的奶牛。特别地，若原图连弱连通分量都不是，那么答案自然是0，否则答案为找到的连通块大小。

解决图论问题的关键是什么？

是算法吗？

建图技巧

HDU2680 Choose the best route

给定含 n 个点 m 条边的有向图，s 个起点，一个终点。求所有起点到终点的最短路中最短的一条。

$n < 1000, m < 20000, 1 \leq s \leq n$

建图技巧

HDU2680 Choose the best route

给定含 n 个点 m 条边的有向图，s 个起点，一个终点。求所有起点到终点的最短路中最短的一条。

$n < 1000, m < 20000, 1 \leq s \leq n$

终点看作起点，起点看作终点，求单源最短路

建立超级源点

建图技巧

Codeforces 186B Legacy

有 n 个点 q 个连边计划。连边计划有三种：1. 给定点 u 向点 v 连边。2. 给定点 u 向编号为 [l, r] 的所有点连边。3. 编号为 [l, r] 的所有点向给定点 u 连边。求给定两点的最短路。

$1 \leq n, q \leq 10^5, 1 \leq s \leq n$

建图技巧

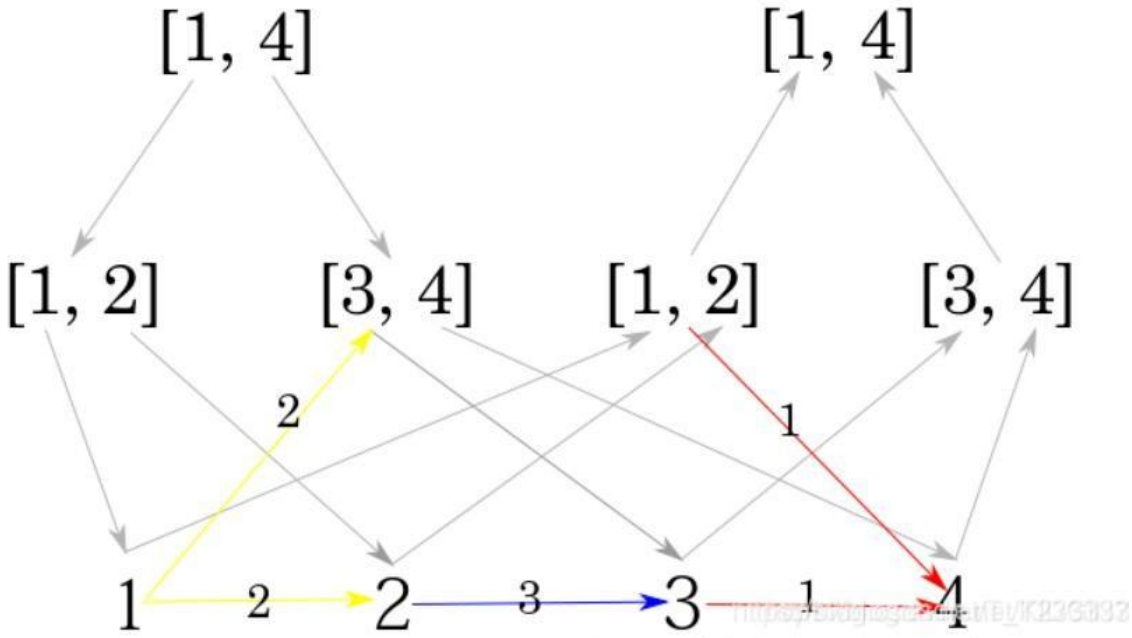
Codeforces 186B Legacy

有 n 个点 q 个连边计划。连边计划有三种：1. 给定点 u 向点 v 连边。2. 给定点 u 向编号为 [l, r] 的所有点连边。3. 编号为 [l, r] 的所有点向给定点 u 连边。求给定两点的最短路。
$1 \leq n, q \leq 10^5, 1 \leq s \leq n$

线段树建图

建图技巧

这个图是进行了三个操作：
节点2到节点3有一条权值为3的边（蓝色）
节点1到区间[2,4]有权值为2的边（黄色）
区间[1,3]到节点4有权值为1的边（红色）



在建树的过程中就让每个线段树节点和他们的叶子节点之间含有一条权值为0的有向边。

之所以是有向边是因为从点到区间的关系有两种，我们得建立两种线段树，分别是从父节点指向叶子节点和从叶子结点指向根节点，对应的是从点到区间和从区间到点的关系。

而且两个线段树的叶子节点，也就是最下面的节点指向的都是原来区间的节点，这样就在原来的区间上建立了从点到区间再从区间到点的关系

建图技巧

洛谷 P4568 [JLOI2011] 飞行路线

给定 n 个点，m 条边的无向图，边权为 c。给定起点终点求最短路。但是，你可以白嫖最多 k 次。

$2 \leq n \leq 10^4, 1 \leq m \leq 5 \times 10^4, 0 \leq k \leq 10, 0 \leq c \leq 10^3$

建图技巧

洛谷 P4568 [JLOI2011] 飞行路线

给定 n 个点，m 条边的无向图，边权为 c。给定起点终点求最短路。但是，你可以白嫖最多 k 次。

$2 \leq n \leq 10^4, 1 \leq m \leq 5 \times 10^4, 0 \leq k \leq 10, 0 \leq c \leq 10^3$

分层图思想

将图分为 k 层，各层之间根据白嫖方案进行连边

要注意，建的图要与实际问题匹配，即每一条最短路对应一种飞行方案

建图技巧

Codeforces 1634 E

给你m个正整数数组。每个数组的长度都是偶数。

你需要将所有这些整数分割成两个大小相等的两个可重集合L和R，也就是说，每个数组的每个元素都应进入两个集合中的一个(但不能同时进入两个)。此外，对于m个数组中的每个数组，其一半的元素应放入L中其余的应放入R中。

请举例说明这种划分方法，或者确定不存在这种划分方法。

建图技巧

Codeforces 1634 E

输入 #1 复制

```
3
2
1 2
4
1 2 3 3
6
1 1 2 2 3 3
```

输出 #1 复制

```
YES
RL
LRLR
RLLRRL
```

建图技巧

Codeforces 1634 E

给你m个正整数数组。每个数组的长度都是偶数。

你需要将所有这些整数分割成两个大小相等的两个可重集合L和R，也就是说，每个数组的每个元素都应进入两个集合中的一个(但不能同时进入两个)。此外，对于m个数组中的每个数组，其一半的元素应放入L中其余的应放入R中。

请举例说明这种划分方法，或者确定不存在这种划分方法。

我们把这个关系抽象成一个二分图，把数组抽象成左边的点，每一种数字抽象成右边的点。如果一个数组A包含一个数字x，那么A代表的点和x就相当于连了一条边，这样就形成了一个二分图，然后只要能在这个二分图上找到一条欧拉回路，我们就找到了一个合法的划分方法。

这道题的难点就在于能够把题面抽象成图论问题

建图技巧

Codeforces 1468 J

给定一个有 n 个节点，m 条无向带权边的图，和一个参数 k，第 i 条边权值为 s_i。

现在你要保留这个图中的 $n-1$ 条边使得这个图变成一棵树，然后你可以对这棵树上的任意边进行修改，每次修改可以使这个边的权值加上一或减去一。

现在你需要使所有边权的最大值**正好等于** k，求所有保留方案的最小操作数。

T 组询问。

保证初始时给定的图满足任意两个点互相可达，没有重边或自环。

建图技巧

Codeforces 1468 J

给定一个有 n 个节点，m 条无向带权边的图，和一个参数 k，第 i 条边权值为 s_i。

现在你要保留这个图中的 $n-1$ 条边使得这个图变成一棵树，然后你可以对这棵树上的任意边进行修改，每次修改可以使这个边的权值加上一或减去一。

现在你需要使所有边权的最大值**正好等于** k，求所有保留方案的最小操作数。

T 组询问。

保证初始时给定的图满足任意两个点互相可达，没有重边或自环。

假设我们只选边权小于等于k的边，如果可以连成一个联通快，那么我们把k-max(所选边权)作为一个可能的答案，接下来我们考虑往这里加边权大于k的边。

因为要保证操作完最大边权等于k，所以再往图中加大于k的边造成的代价是sum(s_i-k)，所以我们可以把已经连好的联通块视作为点，剩下所有边的边权为s_i-k。跑一遍最小生成树就可以。

其他知识

哈密顿回路（通路，需要状压dp）

差分约束(隶属于图论建模，不涉及新知识点)

2-SAT

竞赛图

二分图匹配

网络流

支配树

……

其他知识

我们在掌握基础算法之后，往往还是不能顺利的解决图论问题，主要可能如下：

1、题面里没有提到图有关的描述，根本想不到是个图论题。

2、隐隐感觉可能能抽象成某个模型，但是还是无法成功用图成功描述题目。

3、成功建立了模型，但是因为其他方面的知识欠缺，可能是dp，或者数据结构相关的知识。

4、这道题考察的知识点不会。

……

THANKS